ETAS RTA-RTE v12.7.2

Status: FINAL

RTE MISRA Compliance Statement

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-RTE 12.7.2 - RTE MISRA Compliance Statement R01 EN – 05.2025

Contents

1 About this Manual 4

2 RTA-RTE MISRA 2012 compliance statement 5

3 MISRA violations allowed by RTE_SWS_RTE 6

3.1 MISRA Directive 1.1 and rules 5.1 to 5.5 6

3.2 MISRA Directive 4.10 and Rule 20.1 6

3.3 MISRA Rule 2.3 6

3.4 MISRA Rule 8.5 6

3.5 MISRA Rule 8.7 6

3.6 MISRA Rule 8.8 6

3.7 MISRA Rule 11.2 and 11.3 6

3.8 MISRA Rule 11.5 7

3.9 MISRA Rule 12.3 7

4 RTA-RTE Implementation-Specific Violations 8

4.1 pointer casting for notification polymorphism 8

4.2 pointer casting for copy polymorphism 9

4.3 naked macro parameters that are type expressions in type declarations . 10

4.4 naked macro parameters that are return-type expressions in function declarations 11

4.5 pointer casting for increment 12

4.6 pointer casting for opaque copy 12

5 ETAS Contact Addresses 14

5.1 Technical Support 14

5.2 General Enquiries 14

1 About this Manual

This document is a supplementary reference guide for ETAS RTA-RTE.

It sets out justifications for known unavoidable cases where the generated RTE code violates the MISRA Guidelines for C.

For each case, a description is given of how the guideline is being violated with a detailed justification indicating why the guideline cannot practicably be adhered to and explaining why the usage case will not be problematic.

Within the generated RTE code there are potentially some unavoidable MISRA violations. Static Analysis tools will typically generate warnings about these, with a reference to a particular source line where the violation is considered to have occurred. In order to indicate that such usages have been considered, the referenced source line will contain a justification tag in the form of a C comment in the following format:

```
/* [MISRA <guideline>] {<ID>} */
```

where:

- `<guideline>` identifies the MISRA guideline being violated
- `<ID>` is a unique numeric case identifier linking to this document

Note that in general a single source line may have multiple tags. In such cases each will be in a separate comment.

This format allows for the following workflow:

- Such strictly-formatted comments may optionally be converted to rule-suppression directives for a particular Static Analysis tool that may be in use.
- These comments can be used to automatically discount messages from the report generated by a Static Analysis tool, leaving only the list of messages that do not have justifications.

WARNING

It is the user's responsibility to ensure that any MISRA violations (justified or not) are valid for their field application.

Within this document, justifications are ordered by `<ID>`.

2 RTA-RTE MISRA 2012 compliance statement

RTA-RTE generates C90 code to implement the RTE specified by its input ARXML files. This generated C will not violate any mandatory or required guidelines (rules or directives) from MISRA C 2012 except for the cases described in this document.

Please report any other non-conformances that are encountered through normal reporting channels.

3 MISRA violations allowed by RTE_SWS_RTE

The AUTOSAR specification documents highlight several ways that their requirements mandate C source code that contravenes various MISRA C guidelines. The violations that are relevant to RTE code are collected here for reference. The paragraphs below refer to the following AUTOSAR documents:

SWS_RTE	Document 084 in AUTOSAR R22-11
SWS_MemMap	Document 128 in AUTOSAR R22-11

3.1 MISRA Directive 1.1 and rules 5.1 to 5.5

C90 only requires that the first 6 characters of an identifier are significant. However, the AUTOSAR naming scheme for both data and functions assumes that the compiler treats the whole identifier as significant. This violates MISRA 2012 DIR 1.1 and rules 5.1 to 5.5. See [SWS_Rte_01168] and *Table C.2* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.2 MISRA Directive 4.10 and Rule 20.1

The memory mapping mechanism used by the generated RTE and described by [SWS_MemMap_00003] in SWS_MemMap violates MISRA 2012 Dir 4.10, Rule 20.1.

3.3 MISRA Rule 2.3

[SWS_Rte_02648] specifies that the generated RTE generates type declarations for "*all of the AUTOSAR data types … irrespective of their use by the generated RTE*". This violates MISRA 2012 rule 2.3 if any of the declared types are unused. See *Table C.1* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.4 MISRA Rule 8.5

The generated RTE may violate MISRA 2012 rule 8.5 to support application header file generation. See *Table C.3* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.5 MISRA Rule 8.7

Where SWCs are delivered as object code with ports that are unconnected during generation time, it may sometimes be the case that functions and objects defined with external linkage are only be referenced from one translation unit. This violates MISRA rule 8.7. See *Table C.8* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.6 MISRA Rule 8.8

Declarations of objects with internal linkage may need to be declared static for measurement and calibration. This violates MISRA rule 8.8. See *Table C.4* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.7 MISRA Rule 11.2 and 11.3

Casting from a pointer-to-a-*data-element-with-status* to a pointer-to-a-*data-element-without-status* may be required for interfacing with

COM. This violates MISRA rule 11.2 and 11.3. See *Table C.6* & *Table C.7* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.8 MISRA Rule 11.5

Casting from a pointer-to-*void* to a pointer-to-a-C-type may be required for interfacing with COM. See *Table C.9* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

3.9 MISRA Rule 12.3

The comma operator is used in the implementation of RTE APIs that have been optimized to a macro, to allow their use in expressions. See [SWS_Rte_01146] and *Table C.5* in Appendix C ("*MISRA C Compliance*") in SWS_RTE.

4 RTA-RTE Implementation-Specific Violations

The following MISRA non-conformances are unavoidable in the RTA-RTE implementation. A detailed analysis is provided for each non-conformance stating why the code is considered functionally correct.

4.1 pointer casting for notification polymorphism

Unique ID

10001

MISRA Guideline

2012:Rule 11.3 [Required]

"*A cast shall not be performed between a pointer to object type and a pointer to a different object type*"

(decidable by single translation unit)

Locations

The reported violations (in each case a cast of the `q` parameter) are in the following functions in `Rte_Lib.c`. These are generated from template.

```
Rte_WriteQueue( Rte_QCmnRefType    q,
                Rte_ConstDataPtr   data )

Rte_WriteQueue_GetProtection(Rte_QCmnRefType q)

Rte_WriteQueue_ReleaseProtection(Rte_QCmnRefType q)
```

Justification

Within some generic queue handling functions there are some casts from the input parameter (`Rte_QCmnRefType`, which is a pointer to a `struct Rte_QCmnType`). This parameter type gives a common interface to handle several different models of queue handling.

The set of possible casts made is as follows:

- `Rte_QRefTaskType` (aka pointer to `struct type Rte_QTaskType`)
- `Rte_QRefREType` (aka pointer to `struct type Rte_QREType`)
- `Rte_QRefMSIType` (aka pointer to `struct type Rte_QMSIType`)

All of these structs have a `Rte_QCmnType` as the first member. The C specification states that a pointer to a struct is identical to a pointer to the first member of that struct. Thus passing a pointer to the `Rte_QCmnType` member of an object of any of these types also identifies a pointer to the containing object itself. The passed-in pointer to the `Rte_QCmnType` can be uncontroversially de-referenced, and this struct contains a member of type `Rte_NotificationType`. The value of this member controls which of the larger structs the `Rte_QCmnRefType` is cast to. Thus by the appropriate value being set in the `Rte_NotificationType` member of the `Rte_QCmnType` struct referenced in the first member of the super-object, the correct behavior within the queue functions is selected, including a cast up to the super-object to access mechanism-specific details for how to handle the associ-

atedcase.

NOTE This justification applies ONLY to these specific instances.

Each instance is annotated with a reference comment as follows:

```
/* [MISRA 2012:Rule 11.3] {10001} */
```

4.2 pointer casting for copy polymorphism

Unique ID

10002

MISRA Guideline

2012:Rule 11.5 [Advisory]

"*A conversion should not be performed from pointer to void into pointer to object*"

(decidable by single translation unit)

Locations

The reported violation is in `Rte_WriteQueue()` in `Rte_Lib.c`. This is generated from template.

Justification

For data inserted into queues, there are 2 models of what might be termed *data persistence*

- In the *non-persistent* model, the data that should be placed into an element in the queue is indicated by the "pointer to data" passed to `Rte_WriteQueue()`
- In the *persistent* model, rather than copy the referenced data into a queue element, *the pointer itself* is simply put into the queue (because the pointer is expected to still point to the relevant data when the queue item is popped, and a pointer is typically cheaper to store than a larger data block).

The MISRA violation here is related to the *persistent* model.

The relevant section of code is as follows. The line to which the MISRA message applies is highlighted here with the annotation "pointer casting for copy polymorphism".

```
if ( q->copy )
{
   /* queue keeps a copy of the data as its source does not persist */
   ( void )Rte_memcpy( dynamic->in, data, q->element_size );
}
else
{
   /* queue keeps a reference to the data as its source persists
    * long enough for the item to be consumed */

   // pointer casting for copy polymorphism

   *( P2VAR(Rte_ConstDataPtr, AUTOMATIC, RTE_DATA) )dynamic->in = data;
}
```

The parameter `data` is a pointer-to void to make the interface generic.

When the *persistent* model is in use, the value of the pointer *IS* the data and queue entries will be set up to be the same size as the pointer. The generic queue in-pointer is, however, a pointer-to-void, so it cannot be written through directly (because it is not associated with any idea of object size). Thus in order to write the data-pointer value into the queue through the in-pointer, the in-pointer must be cast to be a pointer-to-pointer.

The switching between the *persistent* and *non-persistent* models of operation is selected by the `copy` member in the passed-in queue handle, as shown above. The value of this member will match the rest of the queue configuration to specify a queue of pointers.

Each instance is annotated with a reference comment as follows:

```
/* [MISRA 2012:Rule 11.5] {10002} */
```

4.3 naked macro parameters that are type expressions in type declarations

Unique ID

10003

MISRA Guideline

2012:Rule 20.7 [Required]

"*Expressions resulting from the expansion of macro parameters shall be enclosed in parentheses*"

(decidable by single translation unit)

Locations

This applies to code widely distributed through generated RTEs, both in code generated algorithmically from the arXML model and in code generated from templates.

For example the source text

```
P2VAR(uint8, AUTOMATIC, RTE_APPL_DATA) DestBuffer,
```

which the pre-processor converts to

```
uint8* DestBuffer,
```

There are many such wrapper macros that take a type name as a parameter and replace with the type name naked in the substitution text.

Justification

In this scenario, it is not possible to comply with the MISRA rule because C language syntax will not allow parentheses around the type mention in a type declaration. For example in the above illustration, adding parentheses as mandated by MISRA would result in the following invalid C content

```
(uint8)* DestBuffer,
```

It is also important to note that in these cases, consecutive tokens on either side of the type mention either from expansion of other macro parameters or from surrounding source code is unlikely to give rise to unexpected evaluation precedence issues (which is the main rationale behind the rule).

NOTE This justification applies ONLY to the following set of wrapper macros, which have been reviewed to be using this idiom.

- VAR
- P2VAR
- P2CONST
- P2FUNC

4.4 naked macro parameters that are return-type expressions in function declarations

Unique ID

10004

MISRA Guideline

2012:Rule 20.7 [Required]

"*Expressions resulting from the expansion of macro parameters shall be enclosed in parentheses*"

(decidable by single translation unit)

Locations

This applies to code widely distributed through generated RTEs, both in code algorithmically generated from the arXML model and in code generated from templates.

For example the source text

```
FUNC(Std_ReturnType, RTE_CODE)
Rte_WriteQueue( ...
```

which the pre-processor converts to

```
Std_ReturnType Rte_WriteQueue( ...
```

Justification

In this scenario, it is not possible to comply with the MISRA rule because C language syntax will not allow parentheses around the type mention of the return type in a function declaration. For example in the above illustration, adding parentheses as mandated by MISRA would result in the following invalid C content

```
(Std_ReturnType) Rte_WriteQueue( ...
```

NOTE This justification applies ONLY to the `FUNC` wrapper macro.

4.5 pointer casting for increment

Unique ID

10005

MISRA Guideline

2012:Rule 11.5 [Advisory]

"*A conversion should not be performed from pointer to void into pointer to object*"

(decidable by single translation unit)

Locations

This applies to the 2 instances of pointer casting in the pointer arithmetic line in each of `Rte_ReadQueue()` and `Rte_WriteQueue()` in `Rte_Lib.c`. These are generated from template.

Relevant lines (after pre-processing):

```
dynamic->out = (char*) ( dynamic->out ) + q->element_size;
  ...
dynamic->in = (char*) ( dynamic->in ) + q->element_size;
```

The `in` and `out` members of `dynamic` (a pointer-to-`Rte_QDynType`) are pointer-to-void.

Justification

In this scenario, the cast of a `void*` pointer to a `char*` pointer is only to indicate that the increment step for pointer arithmetic is `sizeof( char )`, ie the minimum architecture width. The C standard guarantees that pointers to char will not have issues with alignment. In this situation the pointer is never de-referenced and therefore there are no issues with misinterpretation of data semantics due to the cast.

NOTE This justification applies ONLY to specifically identified instances of this kind.

4.6 pointer casting for opaque copy

Unique ID

10006

MISRA Guideline

2012:Rule 11.5 [Advisory]

"*A conversion should not be performed from pointer to void into pointer to object*"

(decidable by single translation unit)

Locations

This applies to the 2 instances of pointer casting in the 2 initialization lines in `Rte_memcpy()` in `Rte_Lib.c`. These are generated from template.

Relevant lines (after pre-processing):

```
      char*  d = (char*)dst;
const char*  s = (const char*)src;
```

Here `dst` and `src` are parameters to the function, with type pointer-to-void.

Justification

In this scenario, the cast of a `void*` pointer to a `char*` pointer is to allow copying of data structures by their smallest architecture word. The data is copied without consideration of the semantics therefore there are no issues with misinterpretation. Also, the data pointers are incremented and casting to the minimum architecture width makes the increment operations step by the correct amount. The C standard guarantees that pointers to char will not have issues with alignment.

NOTE This justification applies ONLY to specifically identified instances of this kind.

5 ETAS Contact Addresses

5.1 Technical Support

Technical support is available to all RTA-RTE users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office (see below).

The best way to get technical support is by email. Any problems or questions about the use of RTA-RTE should be sent to: rta.hotline@etas.com

If you prefer to discuss your problem with the technical support team, you can call the support hotline on: +44 (0)1904 562624. The hotline is available during normal office hours (0900-1730 GMT/BST).

In either case, when contacting technical support, it is helpful if you can provide as much of the following information (where relevant):

- Your support contract number.
- The versions of the ETAS tools you are using.
- The version of the compiler tool chain you are using.
- The command line (or reproduction of steps) that result in an error message.
- The error messages or return codes you received (if any).
- Your .xml and .arxml files.
- Your OS configuration files.
- The diagnostic.dmp file (if it was generated).

5.2 General Enquiries

5.2.1 ETAS Global Headquarters

ETAS GmbH
Borsigstrasse 24
70469 Stuttgart
Germany

Phone: +49 711 3423-0
Fax: +49 711 3423-2106
www: https://www.etas.com

5.2.2 ETAS Local Sales and Support Offices

Contact details for your local sales office and local technical support team (where available) can be found on the ETAS website:

ETAS subsidiaries: https://www.etas.com/en/contact.php
ETAS technical support: https://www.etas.com/en/hotlines.php